Chapter 1 — Dataset formats

Structure tells you how records are organized; format tells you how they are stored

Learning objectives

- Leave able to name formats for tabular, hierarchical
- Geospatial data, explain when CSV, SQL, HDF5, or GeoJSON fits
- Connect format choice to the tools that will consume the data

Why format matters

- Format is the serialization of structure
- Matching format to structure, expected volume
- A mismatch, for example, forcing huge arrays through naive CSV, creates avoidable cost

Common formats at a glance

- CSV remains the common language for simple tables
- SQL tables add typed schemas and efficient queries
- JSON and GeoJSON handle nested objects and spatial features
- HDF5 targets large scientific or array-oriented datasets
- Each aligns with a structural niche

Example 1.8 — SQL format

- Example 1.8 — hands-on module
- Example 1.8 shows structured data in a relational form
- When attributes are stable and analysts need aggregations
- Explore the chapter example module
- View files: `modules/chapter1/example8/`

Example 1.8 — listing

```
]
CREATE TABLE Customers (
    customer_id SERIAL PRIMARY KEY,
    name VARCHAR(100) NOT NULL,
    email VARCHAR(100) UNIQUE NOT NULL
);

CREATE TABLE Orders (
    order_id SERIAL PRIMARY KEY,
    customer_id INT NOT NULL,
    order_date TIMESTAMP DEFAULT CURRENT_TIMESTAMP,
    total_amount DECIMAL(10,2),
    FOREIGN KEY (customer_id) REFERENCES Customers(customer_id)
);

CREATE TABLE Order_Items (
# ...
```

Example 1.9 — HDF5

- Example 1.9 — hands-on module
- Example 1.9 creates and reads an HDF5 file
- Scientific and machine-learning pipelines often prefer HDF5 when CSV becomes too slow or
- Explore the chapter example module
- View files: `modules/chapter1/example9/`

Example 1.9 — listing

```
import h5py
import numpy as np

with h5py.File("example.h5", "w") as hdf:
    data = np.random.rand(100, 100)
    hdf.create_dataset("random_data", data=data)
    group = hdf.create_group("experiment")
    group.create_dataset("temperature", data=np.linspace(0, 100, 100))
    group.create_dataset("pressure", data=np.linspace(1, 10, 100))

with h5py.File("example.h5", "r") as hdf:
    print("Datasets in root:", list(hdf.keys()))
    random_data = hdf["random_data"][:]
    print("Shape of 'random_data':", random_data.shape)
```

Choosing a format

- Start from structure, then consider size, update rate, and tooling
- Prefer formats your team already supports
- Metadata and documentation, covered next, make that choice durable

Takeaways

- Format serializes structure
- Everyday tables lean on CSV and SQL

Next

- Complete the quiz for this part
- Complete the quiz, then move to characteristics of good datasets, accuracy, completeness